

Поиск причины перерисовки

Часто бывает так, что производительность страницы падает из-за лишних синхронизаций, но бывает сложно понять, что именно их вызвало. Например, вы открываете карточку из реестра и у вас перерисовывается реестр.

В этой статье мы научимся находить причины синхронизаций и разберём способы устранения лишних.

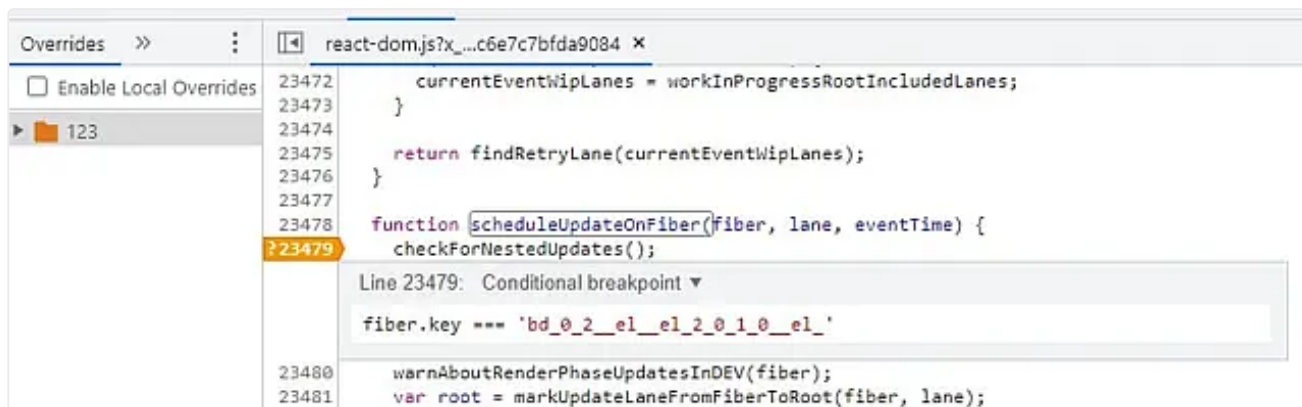
Для начала вспомним следующий факт - **большинство синхронизаций начинаются с изменения состояния**. Также синхронизация может начаться из-за вызова **forceUpdate()** в классовых react-компонентах, но это встречается довольно редко. Изменение состояния может произойти в следующих случаях:

1. Вызов метода **set**, полученного из хука **useState**, в функциональных react-компонентах.
2. Вызов метода **setState** в классовых react-компонентах и **wasaby**-компонентах.

Оба варианта нужно отлаживать по одному и тому же алгоритму:

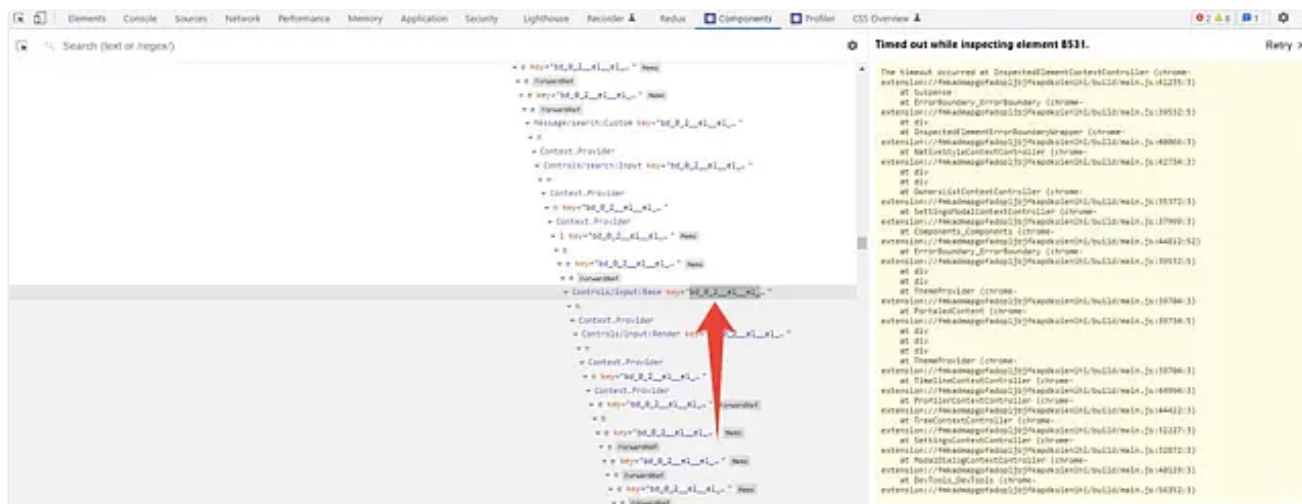
1. С помощью [React Developer Tools](#) снимаем профиль и определяем, какое состояние изменилось. **Не забудьте** поставить галочку "Record why each component rendered while profiling" в настройках профилировщика.
2. Ищем точки изменения этого значения.
3. Ставим в эти места точки остановки, повторяем сценарий, определяем, в какой из точек происходит изменение.
4. Из этой точки поднимаемся по стеку, определяем причину изменения (обычно обработчик события или таймер).

Если точек изменения значения много или вы вообще не можете их найти, то есть вариант поставить [условную точку остановки](#) непосредственно в функцию **scheduleUpdateOnFiber** в файле **react-dom.js**. Чтобы оставить только обновления, относящиеся к вашему компоненту, можно поставить условие на **fiber.key === <ключ_вашего_компонента>**. Если ключа нет, то можно попытаться завязаться на тип компонента, либо его пропсы/состояние.



Ключ можно скопировать, найдя ваш компонент на вкладке Components:

После определения причины изменения исправление обычно сводится к одному из следующих вариантов:



1. Если значение не должно было меняться, нужно добавить проверку перед его изменением. Обычно такое происходит, если в качестве реакции на событие происходит клонирование объекта/массива без просмотра его содержимого.

2. Если значение должно было меняться, но от него зависит несколько контролов, только часть которых должна обновиться, то нужно разделять опции. Например, если есть объект, который передаётся в три контрола, каждому из которых нужна только часть полей этого объекта, то лучше передавать поля по-отдельности, а не весь объект.